CPS610-W26, Assignment-5, Labs 9, 10 – Database Systems-II Due: Apr4, 2026, 4pm

(The TAs will evaluate your work and ask questions about progress and final results in the labs: Weeks of Mar23 & Mar30)

Objectives

- Part1: Installation of MongoDB as a document-based NoSql Database, and Practice with CRUD operations on a different Database architecture
- Part2: Understanding/Implementing Map-Reduce Process on Big-Data resources, and Analyzing Serialized Transactions through Map-Reduce Process

Part1: Instructions to work with NoSql DB: [Total 18 Marks]

In this part, you will install MongoDB which is a document-based and NoSql database on your machine. You will also work with the basic CRUD operations: **Create, Read, Update,** and **Delete** on your installed Mongodb database.

A NoSql database refers to a non-Sql or non-Relational database which uses other mechanisms than relational tables to store and retrieve data. The following indicates various mechanisms that NoSql databases can be created based on:

Document based: Apache CouchDB, Couchbase, IBM, MarkLogic, MongoDB, OrientDB, RethinkDB
Key-value based: Couchbase, Oracle NoSqlDB, Dynamo, Berkley DB, OrientDB
Graph based: Apache Graph, Infinite Graph, Allegro Graph, Neo4J

➢ MongoDB Installation [3 Marks]

Windows Installation

1. Download MongoDB
   - Go to the official site: https://www.mongodb.com/try/download/community
   - Choose:
     - Version: Current
     - Platform: Windows
     - Package: MSI
2. Run Installer
   - Open the .msi file
   - Choose "Complete" setup
   - Check "Install MongoDB as a Service"
   - Optionally install MongoDB Compass (GUI tool)
3. Finish & Verify

Open Command Prompt:
mongod --version

MacOS Installation (using Homebrew)

1. Install Homebrew (if needed)

/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

2. Install MongoDB

brew tap mongodb/brew
brew install mongodb-community

3. Start MongoDB

brew services start mongodb-community

4. Verify

Mongosh

- **CURD Operations -** Practice the following queries after installing MongoDB and add their results and screen-shots in your submission. After running each query (below), relevant results should be displayed by NoSql. [3 Marks]

a) **Create / Insert Operation**

-To Create a new NoSql database named "students"

```
> db.createCollection("students")        ; Create collection
> uses students                          ;Switch to or create database
```

-To insert document into the NoSql MongoDB, using db.<collection>.insert()

```
> db.students.insert ({name: "student1", year: 3, id: "stud1"})
```

-To insert one document into the NoSql MongoDB, using db.<collection>.insertOne()

```
> db.students.insertOne ({name: "student1", year: 3, id: "stud1"})
```

-To insert many documents into the NoSql MongoDB, using db.<collection>.insertMany()

```
> db.students.insertMany ([
{name: "student1", year: 3, id: "stud1"},
{name: "student2", year: 4, id: "stud2"},
{name: "student3", year: 5, id: "stud3"}])
```

-To verify the number of documents in a database, using db.<collection>.count()

```
> db.students.count()
```

b) **Read / Find Operation**

- To Find all documents from the NoSql MongoDB, using db.<collection>.find()

```
> db.students.find ( )
```

- To Find one document from the NoSql MongoDB, using db.<collection>.find() with filters (conditions)

```
> db.students.find ({ name: "student1" })
> db.students.find ({ id: "stud1" })
> db.students.find ({ year: {$gt: 3} })
```

c) **Update Operation**

- To Update one document from the NoSql MongoDB, using db.<collection>.UpdateOne()

Update one:

```
> db.students.UpdatOne ( {name: "student1"},{$set: {year:5}} )
```

- Update many:

```
> db.students.updateMany( { year: { $lt: 3 } }, { $set: { status: "junior" } })
```

- Add new field:

```
> db.students.updateOne( { name: "student2" }, { $set: { graduated: "false" } })
```

- Increment value:

```
> db.students.updateOne( { name: "student3" }, { $inc: { year: 1 } } )
```

d) **Delete / Remove Operation**

-To Remove documents from the NoSql MongoDB, using db.<collection>.remove()

Delete one:

```
> db.students.deleteOne({ name: "David" })
```

Delete many:

```
> db.students.deleteMany({ age: { $gt: 22 } })
> db.students.remove ( {  id: 'stud2' } )
```

Delete all:

```
> db.students.deleteMany ({})
```

e) **Extra Useful Operations**

Count: `> db.students.countDocuments()`

Sort: `> db.students.find().sort({ age: -1 }) // descending`

Limit: `> db.students.find().limit(2)`

- **Questions:** Considering the "Users Collection Dataset", answer the following questions by providing the required queries, relevant results and proper screen-shots in your submission: [12 Marks, 1 Mark each step]

Dataset (Users Collection):
("Alice White", 28, "alice@example.com", "Toronto", true, ["reading", "yoga"]),
("Bob Black", 34, "bob@example.com", "Vancouver", false, ["gaming", "hiking"]),
("Charls Brown", 22, "charls@example.com", "Montreal", true, ["music", "sports"]),
("Diana Blue", 30, "diana@example.com", "Ottawa", true, ["fitness", "travel"]),
("Ethan Red", 40, "ethan@example.com", "Calgary", false, ["climbing", "shooting"]);

Q1- Create a new MongoDB database named "users_xx", (xx is your Team#)

Q2, Q3- Insert values from Dataset "Users Collection" above in your database based on these data types:
(name: varchar(20), age: integer, email: varchar(30), city: varchar(10), isActive: boolean, hobbies: Array of varchar(30));

Q3- Find the list of Active users.

Q5- Find users older than 30.

Q6- Find users in Toronto.

Q7- Update the age for user "Alice White" to 29.

Q8- Set "inactive" users to "Active" users.

Q9- Add a new hobby for the user "Bob Black" as "coding".

Q10- Remove the users younger than 25 year-old.

Q11- List all documents sorted based on the "Age" in descending order.

Q12- Define number of inactive users.

Useful Resources (for Part-1):
https://www.mongodb.com/docs/manual/installation/
https://www.mongodb.com/docs/manual/crud/
https://www.mongodb.com/docs/manual/core/transactions-operations/
https://www.geeksforgeeks.org/mongodb/mongodb-crud-operations/

Part2 - Implementing Map-Reduce Process on Databases [Total 22 Marks]

In the context of Database Management Systems (DBMS), many problems can be processed using the Map-Reduce solution pattern, and in a bigger scale this process can simplify the Big-Data analysis greatly. The Map-Reduce process consists of three major tasks in general:

- Map: Extract something of interest from each record in database
- Shuffle: Group the intermediate results in a specific structure
- Reduce: Applying further process such as summarize /transform on each group of data to find the final result

More specifically, a Map-reduced process generally contains the following functions:

Splitting: The function Split( ), extracts each record from each data file, the required data from that record, and then stores the data. In this function, the data can be stored either individually or in a structure of initial `<key, value>` pair with the value=1. Using the latter case, the whole data will be stored in a collection of initial `<key, value>` pairs

Record of data → collection of (K_i, V_i)

Mapping: The function Map( ), transforms all initial `<key, value>` pairs (for each data file) to the intermediate `<key, value>` pairs

$$(K_1, V_1), (K_1, V_2), \ldots \rightarrow \text{list } (K_1, V_u)$$

Shuffling: The function Shuffle( ), transforms the intermediate <key, value> pairs through sorting, analyzing and classifying process in order to be transferred to the next phase

$$\text{list } (K_1, V_u) \rightarrow (K_1, \text{list}(V_u))$$

Reducing: The function Reduce( ), transforms all intermediate values associated with the same key to a processed value for that key

$$(K_1, \text{list}(V_u)) \rightarrow \text{list } (K_1, V_p)$$

- **Implementation:** In this part, you will simulate a Map-Reduce process on a pre-defined database. Your program to simulate the Map-Reduce process should be implemented using an object-oriented language, based on the following requirements:

a) [8 Marks]

Expected input to the process: Use a **sample** of the huge data file with millions of lines of English Text (just a **sample** of it). This text contains the English words from the reviewers about the nominated movies for the Oscars 2024. Sample data-files containing relevant information for10 nominated movies have been collected and attached (*data-files 1-4*). Assuming the database is distributed on different machines (max 4 machines) physically located at 4 different points. A Map-Reduce process will be used to collect the required data for all movies (#10) from this database.

Change the format of your sample input data (collected from 4 points) to be stored in hash-based arrays (here you need 4 different arrays). Each element in the array can contain a header node pointing to a linked-list. Each node in that linked-list should contain at least 3 fields: the key (movie name), some definition about that key (movie info), and pointer to the next node. The number of the nodes in each linked-list determines the number of the key appeared in its relevant data-file. For example if the linked-list for the key "movie1" from data-file1 has 6 nodes, this means that "movie1" has appeared in its relevant data-file1 for 6 times (every time could have a different definition). The hash-function to address/find each element in the array works based on the input key and your own design.

So at this point, you should have 4 hash-based arrays containing the data from 4 different data-files (attached).
Note-Fell free to design your hash-table, hash-function, and linked-list structure as required in your program.

Expected final output in this process: a list of (key, count) pairs, where the count is the number of occurrences of each key in the whole database (stored on different 4 machines), produced by applying a Map-Reduce process on the input database.

b) [14 Marks]

Implement the Map-Reduce process on the whole database above (4 data-files which are now available in 4 hash-based arrays), with major functions stated above, including:

`map( word, data-file(i))` → `<word, list of counts>`

Given the words in data-file(i), find the list of pairs `<word, count>` from data-file(i) (can be found from the hash-array already implemented in part a, relevant to data-file(i)).

`reduce <word, list of counts>` → `(word, total-count)`

Given the list of pairs `<word, count>`, find the output (w, s) where s is sum of counts for any word.

Note-Each run of your program is only about one input key to be processed in your database, and so word can be any word in data-file (i). But for this program only work on the word = "movie name".
Note-Feel free to use any number of other variables and functions required in your implementation.

Bonus Part (optional)- Using the first version of your program (code completed until after b) and mostly with respect to the Map-Reduce process, which parts of your implemented process can be run concurrently?
In case you find any parts, can you define the whole process in N# of serialized transactions (to be run at the same time with no conflicts)? What is N in your plan? [2 Marks]

- **Optimization:** In order to optimize the implementation of the Algorithm in A5, you can follow the instructions below:

Input-phase:
-Get input text files from various locations (#4): Input1, Input2, Input3, Input4

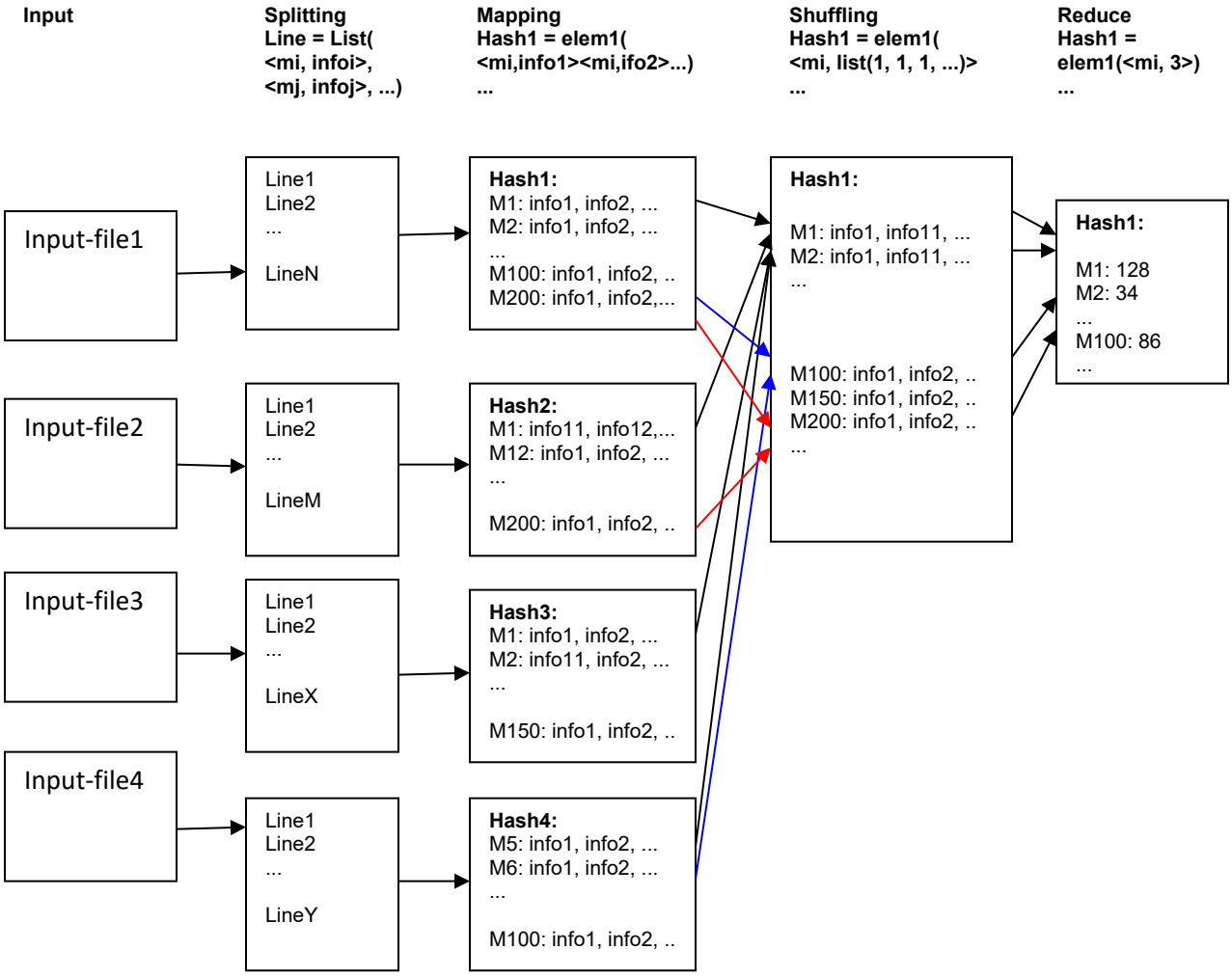
Split-phase:
-Split Input1 to lines: line1, line2, line3, ..., lineN
-Split Input2 to lines: line1, line2, line3, ..., lineM
...
-Split Input4 to lines: line1, line2, line3, ..., lineY

Mapping-phase:
a) Process Input1 (data file1), lines 1-N, insert required data into Hash1, based on <"movie-name", "movie-info">
Extract the data from Input1-Line1 and insert the movie info into the Hash1 table at different elements, for example: if "movie-name1" is found in the Line1, then its corresponding "movie-info" will be added to the Hash1 table and if there are more than one occurrence for "movie-name1" their "movie-info" will be added to that Hash1 element as a linked-list (at the same key "movie-name1"), and this will be repeated for all other lines in the Input1

-Process Input1-line2, insert required data into Hash1, based on <"movie-name", "movie-info">
...
-Process Input1-lineN, insert required data into Hash1, based on <"movie-name", "movie-info">

b) The same as step (a) for all other Inputs 2-4 (data files 2-4) to create hash tables Hash2 to Hash4
Note- At this phase you might add other information like an Index to each node to be saved in its linked-list, for example every time a new info is found in Input1 for "movie-name1", it can be added to the relevant linked-list (key with "movie-name1") with two fields as a pair containing the <"movie-info", "movie-Index">, where Index can be its counter OR you might want to just have one single data "movie-info" (not a pair)

Shuffling phase:
At this phase all 4 Hash tables you have created so far should be merged together as one Hash table; Select one of them as the major Hash like Hash1, and merge the other tables into this major table Hash1:
Go through all elements in the other Hash2 to Hash4 and add their elements to the corresponding elements in Hash1 (for the same key or movie name in their corresponding linked-list)
Note-At this phase you might need to maintain the nodes' structure as a pair of data or not, depending to your own design.
Reducing phase:
a) At this phase for the major Hash1 table, go with each element, counts its nodes in its corresponding linked-list (or if you already had the Index as the counter, use the last Index in the list showing the last count of that key)
b) Create a new Hash table, where each element with the key "movie-name" contains the result of counting that key in the Hash1 as the pair of <"movie-name", "count">
Note-Instead of creating a new Hash table, you might add this result node for each element either at the end or at the beginning of its relevant linked-list in the Hash1 table again

Deliverables & Marking Schema [Total 40 Marks = weight 4%]
- Students are required to go through the instructions above, discuss their solution via team work, and finally submit their solution through D2L by the due date. Only ONE submission is required for each team.
- The TA will mark assignments by evaluating the individual team members through the Q/A during the lab time (in the weeks of March 23&30). Lab attendance is mandatory during those weeks, to get the individual evaluation marks.
- The marks for this Assignment contains two parts:
  1) Individual evaluation marks (only for attended members in the Labs during the weeks of March 23&30) [30 Marks]
  2) Final submission evaluation marks (for the last submissions by Apr 4) [10 Marks]

Useful Resources (for Part-2):
https://www.javatpoint.com/mapreduce
https://www.talend.com/resources/what-is-mapreduce/
https://www.ibm.com/topics/mapreduce
https://stackoverflow.com/questions/